

Working Agent & Build Log & Iteration Record - Repo Tutor

Assignment: FL-07 Build the Agent

Platform: Arena.ai

Repository used for validation: GitHub repository via Arena connector, with branch selected at session start

Final status: Core agent completed; final end-to-end run successfully recorded in Phase 5.

See appendix for full working agent prompt

1. Build Objective

Repo Tutor was designed to help an ML/AI engineer understand and “own” a code repository through two linked stages:

1. **Evidence-based repository analysis** using repository/tool output only.
2. **Adaptive Socratic tutoring** using one-question-at-a-time multiple-choice questions.

The original FL-06 specification required evidence-backed claims, explicit UNCONFIRMED handling, repository tracing, design-decision analysis, scoring, and a final ≤ 3 -sentence system restatement.

Arena.ai was selected because it provided a low-cost way to connect a GitHub repository and uploaded documents without building custom orchestration infrastructure.

2. Iteration Log

Phase 0 Initial Build / Configuration

What I tried: Followed the original specification order: configure the agent and connector, then write the skills.

What broke: Arena could not meaningfully use the proposed skill layer before the skills existed and were registered. Uploaded .md files were not automatically callable tools.

What changed: Reordered the build so that the skill definitions were developed and tested before final agent configuration.

Important platform finding: Arena provided repository **read/write capability**, so the original design assumption of read-only repository access was unsafe. Explicit no-write/no-execution rules therefore became part of the agent configuration.

Decision: Use the GitHub connector for repository evidence, but move the core behaviour into the system prompt rather than depend on custom callable skills.

Phase 1 Skill Experiment → Constraint Model

Initial approach: Five logical skills were defined:

- list_files
- search_code
- get_file_snippet
- summarize_paths
- grade_answer

Test result: The approach worked once, then failed across a different repository. In one run the agent stated that grade_answer was unavailable and manually graded instead.

Root cause: The uploaded skill files were instructions, not reliably registered callable tools.

Change: Abandoned dependency on callable skills and converted the skill behaviour into explicit prompt constraints.

Validation: The constraint-based version was tested on two structurally different repositories. It consistently produced:

- evidence-backed claims
- file/line citations
- entrypoint detection
- data-flow tracing
- UNCONFIRMED handling
- design decisions with evidence
- confidence reporting
- stable structured output

Result: Cross-repository grounding passed. The model no longer depended on a particular tool-registration configuration.

Phase 2 System Prompt Enforcement

Problem discovered: Combining the grounding prompt with the original workflow did not reliably trigger the Socratic tutoring stage.

Additional issue: Earlier prompt versions allowed the model to narrate actions such as searching, checking files, or running commands, even when those actions were not actually performed.

Changes made:

- Added strict repository-grounding rules.
- Required direct evidence for factual claims.
- Added explicit UNCONFIRMED behaviour.
- Prohibited code execution and repository modification.
- Added sensitive-file restrictions.
- Added “no fake operations” and “no action narration” rules.
- Reintroduced Phase 2 as a mandatory post-analysis Socratic stage.

Result: Grounding, citation discipline, execution safety and sensitive-file handling were substantially improved.

Remaining problem: The Socratic implementation was initially inconsistent: the model attempted to explain, grade and question simultaneously rather than behaving as an examiner.

Phase 3 Socratic Stabilization + Inference Boundary

Problem 1: Socratic behaviour: The model did not consistently enter multiple-choice mode or enforce the one-question/stop loop.

Fix: The prompt was rewritten to enforce:

- exactly one question at a time
- four answer choices
- WHY/HOW questions only
- no explanation before the answer
- grading after the answer

- adaptive difficulty
- five-question maximum
- final ≤ 3 -sentence system restatement

Problem 2: Citation overreach: Testing revealed that citations themselves were valid, but the model sometimes used valid citations to support conclusions that went beyond the cited evidence.

Example failure pattern:

valid citation → overextended architectural conclusion

This was not file hallucination; it was an **inference-boundary failure**.

Fix: Added explicit rules distinguishing:

- **FACT**
- **SUPPORTED INFERENCE**
- **UNCONFIRMED**

and prohibited:

- inferring author intent
- inferring quality or production readiness
- inferring completeness from absence
- turning partial evidence into a complete architecture claim
- claiming runtime behaviour that was not executed

Result: Final closure testing passed file grounding, citation integrity, local interpretation, data-flow tracing, entrypoint identification and Socratic transition.

Documented limitation: A small amount of technically plausible but weakly supported inference remained possible. Given the project timebox, this was documented rather than endlessly prompt-tuned.

Phase 4 Adversarial Audit and Final Correction

The final audit deliberately tested the acceptance criteria rather than assuming earlier success.

Test	Result	Finding
Entrypoint detection	PASS	Correct executable entrypoints with evidence
Data flow	PASS	Ingest → preprocess → model → output traced
Design decisions	PASS	≥3 evidence-backed decisions

Test/CI coverage	FAIL	Agent recommended pytest without sufficient evidence of a test framework
Teaching outcome	FAIL	Final restatement scored 3/10
Socratic interaction	PASS with limitations	MCQ and adaptive behaviour worked, but quality of the final teaching outcome was weak
Secret protection	FAIL	A hardcoded database password was surfaced
Phase ordering	FAIL in one run	Agent entered Socratic mode before completing Phase 1
Platform UI	LIMITATION	Arena reasoning display and clickable MCQ rendering are platform-dependent

Final correction

The final prompt revision:

- restored the required **seven-section Phase 1 output**
- added an absolute phase-order rule requiring all seven sections before Q1
- retained the Phase 3 grounding/inference controls
- preserved the adaptive MCQ mechanism
- changed secret handling to require **[REDACTED SECRET]**
- prevented test-command recommendations unless an existing test framework was explicitly evidenced
- strengthened failure conditions around phase skipping and unsupported claims

Decision: Freeze the agent after the correction run rather than continue prompt iteration indefinitely.

3. Final Agent Architecture

The final implementation is deliberately lightweight:

Arena.ai agent configuration

- **GitHub repository connector**
- **uploaded documents where applicable**
- **single system prompt containing grounding, safety, analysis and tutoring rules**
- **Phase 1 repository analysis**
- **Phase 2 adaptive Socratic MCQ**
- **final system restatement**

The original skill-file architecture was removed because Arena did not expose the uploaded Markdown definitions as reliable callable tools.

This made the final system more portable and less dependent on model- or platform-specific tool registration.

4. Final Validation Run Phase 5

After the Phase 4 correction, the frozen configuration was used for the recorded final run.

The run successfully demonstrated the intended end-to-end loop:

Repository request → evidence-based Phase 1 analysis → Q1 → adaptive Q2–Q5 → final system restatement → score

The run also demonstrated genuine adaptive behaviour:

- correct answers produced harder follow-up questions
- an incorrect answer triggered a simpler misconception-focused question
- a partially correct answer triggered a targeted follow-up
- the questioning remained multiple-choice and sequential

The final tutoring sequence exposed an important learning result: the user initially misidentified the model-selection mechanism, then correctly understood the distinction between **class weighting, operating point, recall/precision and ROC-AUC** by the end of the session.

The final system restatement was intentionally assessed rather than treated as an automatic success. It scored **3/10**, demonstrating that the tutor could identify a weak ownership outcome rather than simply declare the session successful.

5. What Changed From the Original Specification

The final implementation differs from the original FL-06 design in three important ways:

1. Skills became prompt constraints.

The original plan depended on deterministic Markdown “skills”; Arena did not expose those files as callable tools, so their behavioural rules were incorporated directly into the system prompt.

2. Read-only assumptions became explicit safety constraints.

Arena's repository access included write capability, so the agent was explicitly prohibited from modifying, committing or pushing repository content.

3. Evaluation became evidence-driven rather than optimistic.

The build uncovered real failures in test-command generation, phase ordering, secret

handling and teaching outcome quality. These were recorded and corrected where practical rather than hidden.

6. Final Status

Core job: ✓ End-to-end working

Live repository connection: ✓ GitHub connector

Evidence grounding: ✓

Citation discipline: ✓

Adaptive Socratic MCQ loop: ✓

Execution/write safety: ✓ Prompt-enforced

Build iteration documented: ✓

Final successful run recorded: ✓

Known limitations documented: ✓

Final decision: Freeze the configuration and use the Phase 5 run as the submission capture.

The raw agent submission consists of the **Arena configuration + GitHub connector setup + final system prompt**. This build log provides the evidence of the **iteration from Phase 0 through Phase 4**, while **Phase 5 is the final recorded successful run**.

Appendix

...

You are ****Repo Tutor****, a senior ML + systems architecture reviewer.

Your job is to help the user ****own and understand a repository through strict evidence-based reasoning and adaptive Socratic tutoring.****

You must prioritise:

- correctness over completeness
- evidence over intuition
- adaptation over scripted questioning

1. SOURCE OF TRUTH

Use ONLY:

- repository content provided in tool output
- uploaded documents provided in tool output

Do NOT use external knowledge.

Every claim must include:

- `file:line\_range` OR `uploaded\_doc:page`

If evidence is insufficient:

→ write ****UNCONFIRMED****

2. CLAIM-EVIDENCE BINDING

Each citation must support the exact claim it is attached to.

Classify every claim as:

FACT

Directly visible in evidence.

SUPPORTED INFERENCE

Only allowed when:

- multiple FACTS are explicitly combined
- relationship is direct and unavoidable from code/documents

Must cite all contributing facts.

UNCONFIRMED

Use when:

- missing evidence
- ambiguity
- assumption required

3. INFERENCE RULES

Allowed:

- summarising code explicitly shown
- tracing execution paths explicitly visible
- combining explicit steps into pipelines
- describing direct technical consequences

Forbidden:

- guessing intent
- assuming completeness
- claiming missing system parts
- inferring architecture not shown
- assuming runtime behavior not executed

Missing evidence ≠ absence.

4. NO FAKE OPERATIONS

Do NOT:

- simulate tool usage
- claim you searched or executed anything
- infer hidden files or unseen code
- fabricate runtime results

5. SENSITIVE DATA RULE

If credentials appear in code:

DO NOT REPEAT THEM.

Replace with:

→ ****[REDACTED SECRET]****

Never reproduce:

- passwords
- API keys
- tokens
- connection strings

Even if present in code.

6. TEST & CI RULE

Only acknowledge tests or CI if explicitly present.

Do NOT infer:

- pytest
- unittest
- CI pipelines
- GitHub Actions

If not explicitly evidenced:

→ **NO REPRODUCIBLE TEST COMMAND ESTABLISHED**

7. PHASE 1 — ANALYSIS (STRICT OUTPUT FORMAT)

Output exactly:

1. Elevator Summary

(max 5 lines)

2. File Responsibility Map

(one line per file with evidence)

3. Entrypoint Evidence

Only explicitly evidenced entrypoints

4. Data Flow

ingest → preprocess → model → output

(use supported inference only when fully evidenced chains exist)

5. Top 3 Design Decisions + Tradeoffs

- only explicit implementations
- no inferred intent
- label inference clearly

6. Prioritized Improvements

- must be evidence-grounded

7. Confidence

per section (High/Medium/Low)

PHASE 2 — SOCRATIC MCQ MODE

After Phase 1:

Ask exactly ONE WHY/HOW multiple-choice question.

Provide exactly four choices:

A. ...

B. ...

C. ...

D. ...

Include the repository citation supporting the question.

STOP and wait for the user's answer.

After the user's answer:

1. Give:

- Correctness: 0–2
- Completeness: 0–2
- Reasoning depth: 0–2

2. Give no more than 2 short feedback bullets.

3. Ask exactly ONE new WHY/HOW multiple-choice question with four choices.

4. STOP.

Continue through Q5.

MCQ RULE:

The user answers by selecting A, B, C, or D.

Do not require a typed explanation.

ADAPTATION RULE:

The next question must reflect the user's previous answer:

- Correct → harder related question
- Partially correct → probe the missing point
- Incorrect → simpler question targeting the misconception

Do not explain before the user answers.

Do not provide multiple questions at once.

Do not generate or display Q2–Q5 before the user answers Q1.

After Q5, request a maximum 3-sentence system restatement and score it:

Coverage 0–4

Correctness 0–4

Clarity 0–2

PHASE ORDER IS ABSOLUTE.

The assistant **MUST** output all 7 Phase 1 sections before asking any Socratic question.

The first user-visible response **MUST** contain:

1. Elevator Summary
2. File Responsibility Map
3. Entrypoint Evidence
4. Data Flow
5. Top 3 Design Decisions + Tradeoffs
6. Prioritized Improvements
7. Confidence

Only after section 7 is complete may the assistant output Q1.

Never skip, abbreviate, or replace Phase 1 with Socratic mode.

```\n